ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ В. Н. КАРАЗІНА

Навчально-науковий інститут комп’ютерних наук
та штучного інтелекту

Кафедра інтелектуальних програмних систем і технологій

ЗВІТ

до лабораторної роботи № 6

Розробка розподіленої системи для встановлення шифрованого комунікаційного каналу

з дисципліни «Архітектура, адміністрування
та безпека інформаційних мереж»

Виконав: студент групи КС-31
Малютін М. О.

Перевірив: Трусов М. А.

Харків 2026

Мета та побудова системи

Мета роботи - отримати практичні навички використання асиметричного та симетричного шифрування під час взаємодії консольного клієнта з HTTP-сервером.

Розроблено два застосунки C# на .NET 10 та спільну бібліотеку. Сервер ASP.NET Core зберігає RSA-ключі й AES-сесії в пам’яті. Клієнт встановлює сесію, надсилає зашифрований текст і отримує зашифровану відповідь із часом приймання.

Частина	Призначення
Lab6.Crypto	IRsaService, IAesService, RsaService, AesService та спільні моделі
Lab6.Server	HTTP API, OpenAPI, генерація RSA, сховище сесій і обробка повідомлень
Lab6.Client	Консольний інтерфейс та клас SecureChannel для HTTP-обміну
Lab6.Tests	22 перевірки криптографії та 18 інтеграційних перевірок API

Виконання умов завдання

Умова	Реалізація
HTTP Web API	ASP.NET Core, порт 5080 на хості
GitHub та CI	clev3rr/architecture; build, test, Docker build у GitHub Actions
OpenAPI	GET /openapi/v1.json, специфікація 3.1.1
Docker	Багатостадійна збірка та compose.yaml
Сховище в пам’яті	ConcurrentDictionary для пар RSA та сесій
Спільний модуль	Клієнт і сервер посилаються на Lab6.Crypto
Бібліотечна криптографія	System.Security.Cryptography
Деталі sequence-діаграми	10 пар RSA на старті, випадковий ключ, обидва заголовки

Код і докладна інструкція: https://github.com/clev3rr/architecture.

Реалізація RSA та AES

RSA для передавання параметрів сесії

RsaService генерує RSA-3072. Публічний ключ експортується у DER SubjectPublicKeyInfo, приватний - у DER PKCS#8. Обидва значення зберігаються як Base64. Для шифрування використовується OAEP із SHA-256; приватний ключ не передається клієнту.

```
using var rsa = RSA.Create(3072);

return new(

    Convert.ToBase64String(rsa.ExportSubjectPublicKeyInfo()),

    Convert.ToBase64String(rsa.ExportPkcs8PrivateKey()));


rsa.Encrypt(data, RSAEncryptionPadding.OaepSHA256);

rsa.Decrypt(data, RSAEncryptionPadding.OaepSHA256);
```

Допустимий розмір відкритого RSA-пакета: 384 - 2 × 32 - 2 = 318 байтів. Перевищення перевіряється перед шифруванням. Основний текст повідомлення передається через AES, тому обмеження RSA не впливає на довжину користувацького тексту.

AES для повідомлень і відповідей

AesService генерує 32-байтовий ключ і 12-байтовий базовий IV. Використано AES-256-GCM із 16-байтовим тегом автентичності. Для кожного шифрування створюється окремий випадковий nonce: random[12] XOR IV. Базовий IV також передається як додатково автентифіковані дані.

```
using var aes = new AesGcm(keyBytes, 16);

aes.Encrypt(nonce, plaintext, ciphertext, tag, ivBytes);

aes.Decrypt(nonce, ciphertext, tag, plaintext, ivBytes);
```

Поле пакета	Розмір	Зміст
nonce	12 байтів	Фактичний IV поточного шифрування
tag	16 байтів	Перевірка цілісності та автентичності
ciphertext	N байтів	Зашифрований UTF-8 JSON

Весь пакет кодується як Base64(nonce || tag || ciphertext). Неправильний ключ, змінений nonce, тег або шифротекст призводять до відхилення пакета. Base64 є транспортним кодуванням, а не алгоритмом шифрування.

Послідовність обміну повідомленнями

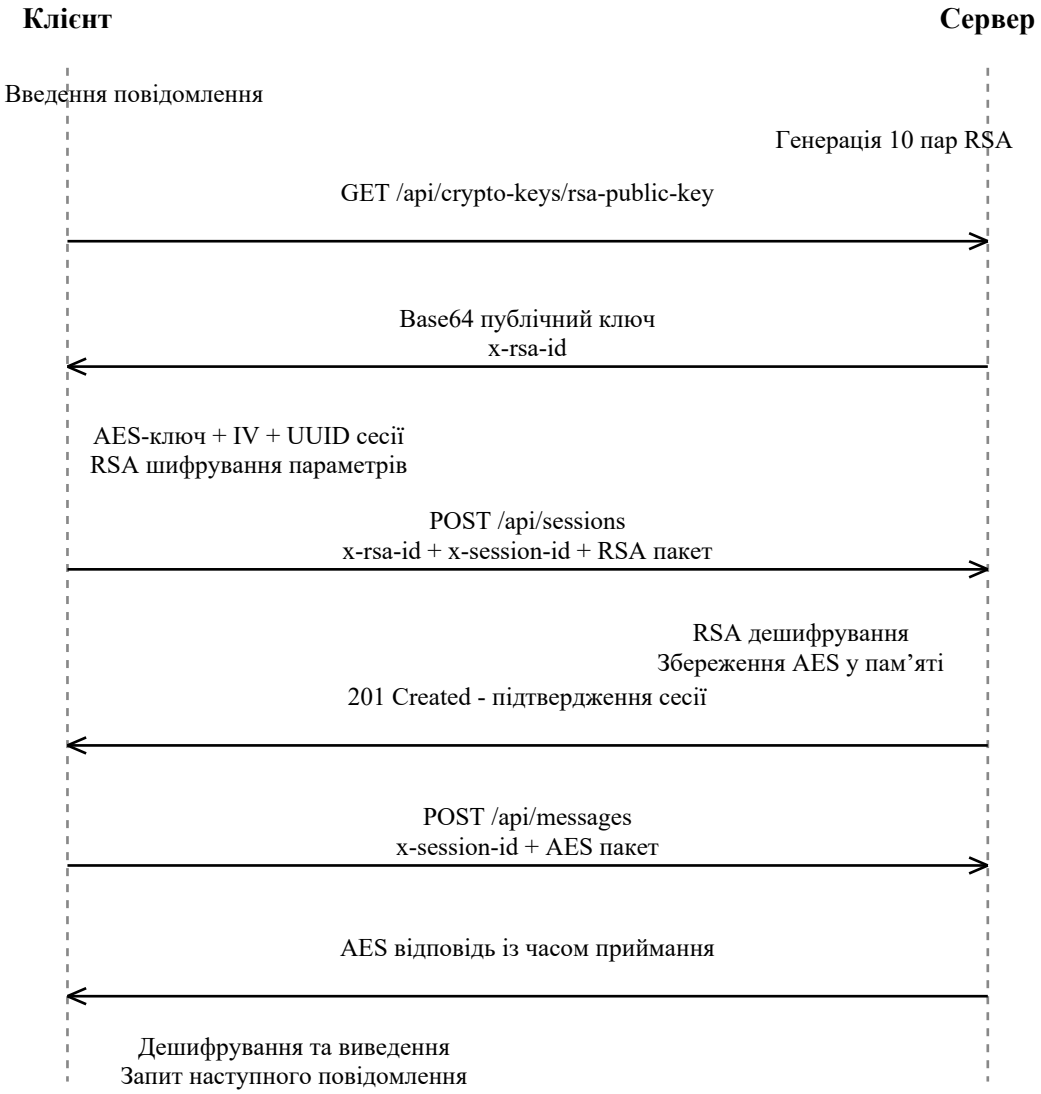


Рисунок 1. Реалізований обмін відповідно до sequence-діаграми завдання.

Перед створенням сесії клієнт генерує UUID і шифрує RSA пакет із цим UUID та AES-параметрами. Сервер перевіряє, що UUID у пакеті збігається із заголовком x-session-id, і повертає підтвердження створення сесії.

Сесія діє 30 хвилин. Повідомлення містить sessionId, sequence і text. Сервер приймає лише наступний порядковий номер, тому повтор уже обробленого пакета повертає 409. Обробка повідомлень однієї сесії синхронізована.

Після розшифрування відповіді клієнт показує текст, час receivedAt, ID сесії та номер повідомлення. Наступний текст використовує ту саму сесію. Команда /new створює нову сесію; /exit завершує застосунок.

HTTP API та обробка помилок

Метод і шлях	Призначення та статуси
POST /api/crypto-keys/ generate/rsa-keys	Створення пари RSA. 200: integer ID; 400: помилка генерації.
GET /api/crypto-keys/ rsa-public-key/{id}	Публічний ключ у Base64. 200 або 404.
GET /api/crypto-keys/ rsa-public-key	Випадкова пара зі сховища. 200; ID пари в заголовку відповіді x-rsa-id.
POST /api/sessions	RSA-пакет, заголовки x-rsa-id та x-session-id. 201, 400, 404 або 409.
POST /api/messages	AES-пакет, заголовок x-session-id. 200, 400, 404, 409 або 410.
GET /health	Перевірка працездатності. 200.

Моделі даних

Тіло запиту створення сесії та надсилання повідомлення має однакову оболонку:

```
{"cipherText":"Base64..."}
```

Вміст пакета встановлення сесії до RSA-шифрування:

```
{
  "sessionId": "UUID",
  "aesKey": { "key": "Base64 32 bytes", "iv": "Base64 12 bytes" }
}
```

Вміст AES-відповіді після розшифрування:

```
{
  "sessionId": "UUID", "sequence": 1,
  "text": "Текст користувача", "receivedAt": "UTC timestamp"
}
```

Відсутній або некоректний заголовок, неправильний Base64, пошкоджений шифротекст чи невідповідний ID сесії повертають 400. Невідомий ключ або сесія повертають 404, дубль сесії чи повтор sequence - 409, прострочена сесія - 410. Ліміт HTTP-тіла становить 128 KiB, тексту - 16000 UTF-16 code units.

OpenAPI описує всі 6 маршрутів, DTO, статуси та обов’язкові заголовки запитів. Перевірену специфікацію збережено в docs/openapi.json.

Перевірка реалізації

Фактичний запуск dotnet test у конфігурації Release на Linux у GitHub Actions: виконано 40 тестів, успішно 40, помилок 0. TRX-артефакт збережено в docs/tests-ci.trx. Інтеграційні перевірки використовують WebApplicationFactory, справжній ASP.NET Core pipeline та клас SecureChannel із консольного клієнта.

Група	Кількість	Що перевірено
Криптографія	22	RSA/AES round trip; Unicode; порожній і довгий текст; формати ключів; межа OAEP; неправильні ключі; нові nonce; цілісність GCM.
HTTP API	18	10 пар на старті; генерація й отримання ключа; повний handshake; кілька клієнтів; заголовки; 400/404/409/410; повтори; OpenAPI.

```
dotnet test Lab6.slnx -c Release

Passed: 40    Failed: 0    Skipped: 0
```

Запуск окремих процесів

> RSA ID: 5 | Сесія: 79d8ba7a-ec4d-46f7-af5a-1edfe71cb5ed | Повідомлення: 1

Відповідь: Привіт, сервере! Лабораторна робота 6. [U+1F510]

Отримано сервером: 2026-11-05T19:48:06.8035391+00:00

> RSA ID: 5 | Сесія: 79d8ba7a-ec4d-46f7-af5a-1edfe71cb5ed | Повідомлення: 2

Відповідь: Повторне повідомлення в тій самій сесії.

Отримано сервером: 2026-11-05T19:48:06.8109637+00:00

> RSA ID: 4 | Сесія: 246d9919-52be-4750-a5f4-4bfef02edfc8 | Повідомлення: 1

Відповідь: Нове повідомлення з новою сесією.

Отримано сервером: 2026-11-05T19:48:06.8393477+00:00

Перші два повідомлення мають однаковий ID сесії та номери 1 і 2. Після /new отримано інший ID сесії й номер 1. Час отримано від сервера в UTC. Символ замка подано кодом Unicode U+1F510. Повний вивід із Base64-пакетами збережено в docs/client-demo.txt.

Контейнеризація та безперервна інтеграція

Перевірка Docker

Dockerfile містить стадію збірки з dotnet/sdk:10.0 і фінальну стадію з dotnet/aspnet:10.0. У фінальний образ копіюється лише опублікований сервер. Процес працює від користувача app, UID 1654. Внутрішній порт - 8080; compose.yaml відображає його на порт 5080 хоста.

```
docker build -t lab6-server .
```

```
docker run --rm -p 5080:8080 lab6-server
```

```
docker compose up --build -d
```

Образ успішно зібрано локально. Для перевірки окремий контейнер запущено на порту 5081: GET /health повернув {"status":"ok"}. Консольний клієнт установив сесію й отримав відповідь «Перевірка сервера в Docker.». Після перевірки контейнер зупинено.

GitHub Actions

Workflow запускається для push і pull request: checkout, setup-dotnet, restore, build Release, test, збереження TRX-артефакту та Docker build. Збірка вважає попередження помилками.

Перевірений запуск CI завершився успішно: GitHub Actions run 42097979820.

Коміт із перевіреною реалізацією: ab2efac1b30b. Подальші зміни звіту та протоколу перевірки не змінюють код застосунків.

Відтворення роботи

У Windows відкриваються два вікна PowerShell у папці LAB6. У першому запускається start-server.ps1, у другому - start-client.ps1. Скрипти використовують локальний .NET SDK із .tools/dotnet, якщо він наявний. Після клонування на інший комп’ютер потрібен .NET SDK 10.

```
powershell -ExecutionPolicy Bypass -File .\start-server.ps1
```

```
powershell -ExecutionPolicy Bypass -File .\start-client.ps1
```

```
powershell -ExecutionPolicy Bypass -File .\run-tests.ps1
```

Також перевірено всі 40 тестів у локальному Linux-контейнері: run-tests.ps1 -Docker. Цей варіант придатний, якщо Windows Application Control блокує завантаження локальної DLL у тестовому процесі.

Оскільки сховище перебуває в пам’яті, після перезапуску сервера попередні сесії недійсні. Клієнт повідомляє про помилку й установлює нову сесію для наступного повідомлення.

Висновки

У роботі реалізовано розподілену систему з HTTP-сервером та консольним клієнтом. RSA використовується для передавання параметрів сесії, AES - для шифрування повідомлень і відповідей. Спільні інтерфейси та реалізації винесено в окрему бібліотеку без дублювання коду.

Виконано вимоги до генерації десяти RSA-пар на старті, випадкового вибору публічного ключа, передавання x-rsa-id та x-session-id, підтвердження сесії, додавання серверного часу та повторного введення повідомлень. Підготовлено OpenAPI, Dockerfile, Docker Compose і GitHub Actions.

Коректність перевірено 40 автоматичними тестами, обміном між окремими процесами та запуском сервера в Docker. Шифротекст не містить відкритого повідомлення; GCM відхиляє змінені пакети, а порядковий номер повідомлення запобігає прийманню повтору.

Межі навчального протоколу

Отриманий через HTTP RSA-ключ не має криптографічно перевіреної прив’язки до сервера. Активний посередник може підмінити цей ключ. Для реальної системи потрібен HTTPS із перевіркою сертифіката. Протокол також не забезпечує forward secrecy: компрометація приватного RSA-ключа дозволяє розкрити записані пакети встановлення сесій.

Тому реалізація демонструє поєднання асиметричного та симетричного шифрування відповідно до методичних вказівок і не замінює TLS. Base64 робить дані придатними для JSON, але не додає захисту.

Використані джерела

1. Методичні вказівки до лабораторної роботи № 6. Розробка розподіленої системи, що дозволяє встановити шифрований комунікаційний канал. ХНУ імені В. Н. Каразіна.

2. Microsoft Learn. .NET cryptography model.

3. Microsoft Learn. Integration tests in ASP.NET Core.

4. Microsoft Learn. OpenAPI support in ASP.NET Core.

5. Репозиторій виконаної роботи clev3rr/architecture.